VELLORE INSTITUTE OF TECHNOLOGY
(Deemed to be University under Section 3 of UGC Act, 1956)

REGEXLAB: AN INTERACTIVE REGULAR-EXPRESSION-TO-AUTOMATA COMPILER PIPELINE WITH VERIFIED NFA OPTIMIZATION

A Project Report submitted in partial fulfilment of the requirements for the course

BCSE307P – Compiler Design Project

by

Vignesh P C (24BCE0695)

Coppisetty Venkat Sai Ujwal (24BCE0667)

Under the guidance of

Dr. Islabudeen M
Associate Professor Grade 1, SCOPE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October 2026

DECLARATION

We hereby declare that the project report entitled **"REGEXLAB: AN INTERACTIVE REGULAR-EXPRESSION-TO-AUTOMATA COMPILER PIPELINE WITH VERIFIED NFA OPTIMIZATION"** submitted by us to Vellore Institute of Technology, Vellore in partial fulfilment of the requirement for the award of the degree of B.Tech. in Computer Science and Engineering, under the course BCSE307P, is a record of bona fide project work carried out by us under the guidance of Dr. Islabudeen M. We further declare that the work reported in this report has not been submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date: **<<TODO: submission date>>**

Vignesh P C (24BCE0695)

Coppisetty Venkat Sai Ujwal (24BCE0667)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **Dr. Islabudeen M**, Associate Professor Grade 1, School of Computer Science and Engineering (SCOPE), for his valuable guidance, continuous encouragement, and constructive feedback throughout the course of this project.

We are grateful to **Dr. Jaisankar N**, Dean, School of Computer Science and Engineering, for providing the infrastructure and academic environment that made this project possible.

We also thank the faculty of the Compiler Design course (BCSE307P) for the opportunity to apply the concepts of lexical and syntax analysis to a project of our own design, and our families and friends for their constant support.

TABLE OF CONTENTS

DECLARATION **2**
ACKNOWLEDGEMENT **3**
LIST OF FIGURES **5**
LIST OF TABLES **6**
LIST OF ABBREVIATIONS **7**
SYMBOLS AND NOTATIONS **8**
ABSTRACT **9**
1. INTRODUCTION **10**
1.1 Background 10
1.2 Motivations 10
1.3 Scope 10
2. PROJECT DESCRIPTION AND GOALS **11**
2.1 Literature Review 11
2.2 Research Gap 11
2.3 Objectives 11
2.4 Problem Statement 11
2.5 Project Plan 11
3. TECHNICAL SPECIFICATION **13**
3.1 Functional and Non-functional Requirements 13
3.2 Feasibility Study 13
3.3 Hardware and Software Specification 13
4. DESIGN APPROACH AND DETAILS **15**
4.1 System Architecture 15
4.2 Data Flow, Use Case, Class and Sequence Diagrams 15
5. METHODOLOGY AND TESTING **17**
6. PROJECT DEMONSTRATION **19**
7. RESULT AND DISCUSSION **20**
8. CONCLUSION AND FUTURE WORK **22**
9. REFERENCES **23**
APPENDIX A – SAMPLE CODE **24**

LIST OF FIGURES

Figure 4.1 – System architecture (layered) 8
Figure 4.2 – Data flow diagram (Level 1) 9
Figure 4.3 – Use case diagram 10
Figure 4.4 – Class diagram (core data contracts) 10
Figure 4.5 – Sequence diagram for one compile-and-match request 11
Figure 5.1 – Pass pipeline for pattern (a|b)*abb 13

Note: figures in this report are schematic diagrams drawn directly from the project's own source identifiers (file/type/function names verified against the code), not graphviz/matplotlib output. Live browser screenshots of the running application (AST/NFA/DFA/min-DFA views, step playback, Growth Panel, optimizer safe/unsafe witness) were **not captured**, because doing so needs a running dev server driven through a real browser session, which was not done while drafting this report; this is listed here rather than faked.

LIST OF TABLES

Table 3.1 – Functional requirements 6
Table 3.2 – Non-functional requirements 7
Table 3.3 – Hardware and software specification 7
Table 5.1 – Module description per compiler pass 12
Table 5.2 – Complexity of each pass 14
Table 5.3 – Input guards 14
Table 5.4 – Automated test results 15
Table 5.4b – Compiled-WebAssembly limit-probe suite 15
Table 7.1 – Pipeline state counts for representative patterns 17
Table 7.2 – Minimization stress timings, native build 18
Table 7.2b – Same three patterns, rebuilt WASM module 18
Table 7.3 – Bounded-optimizer safe vs. unsafe deletion examples 18
Table 7.4 – Cost analysis (tooling) 19

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ASCII	American Standard Code for Information Interchange
AST	Abstract Syntax Tree
CI	Continuous Integration
CMake	Cross-platform Make (build-configuration tool)
CSS	Cascading Style Sheets
DFA	Deterministic Finite Automaton
DFD	Data Flow Diagram
HTML	Hypertext Markup Language
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LALR	Look-Ahead Left-to-right Rightmost-derivation (parser class)
LL(1)	Left-to-right, Leftmost-derivation, 1 symbol of lookahead
NFA	Nondeterministic Finite Automaton
PR	Pull Request
SCOPE	School of Computer Science and Engineering
SVG	Scalable Vector Graphics
TOC	Table of Contents
TS	TypeScript
UML	Unified Modeling Language
VIT	Vellore Institute of Technology
WASM	WebAssembly

SYMBOLS AND NOTATIONS

Symbol	Meaning
Σ	Input alphabet (the set of symbols the automaton reads; ASCII bytes 0x00–0x7F in this project)
Q	Finite set of states of an automaton
q_0	Start (initial) state, $q_0 \in Q$
F	Set of accepting (final) states, $F \subseteq Q$
δ	Transition function, $\delta: Q \times \Sigma \to Q$ (DFA) or $Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ (NFA)
ε	The empty string; also labels a "no input consumed" transition in an NFA
$L(M)$	The language (set of accepted strings) recognized by automaton M
$\|Q\|$	Number of states (state count)
$2^{\|Q\|}$	Powerset ceiling – the theoretical maximum number of DFA states subset construction could produce from an NFA of $\|Q\|$ states
$O(\cdot)$	Big-O asymptotic time/space complexity notation
$\perp$	The rejecting sink (dead) state shown by the visualizer when a DFA/min-DFA transition is missing

ABSTRACT

Regular expressions are usually presented to students as an opaque, black-box feature of a programming language's standard library, which hides exactly the constructions – Thompson's construction, subset construction, and DFA minimization – that a compiler-design course asks students to learn. RegexLab is a teaching tool that removes that opacity: it is a hand-built, dependency-free regular-expression engine that compiles a supported pattern through an explicit, inspectable pass pipeline – parser, Thompson's construction, subset construction, Moore's minimization, and a matcher – and exposes every intermediate structure (the abstract syntax tree, the nondeterministic finite automaton, the deterministic finite automaton, and the minimized deterministic finite automaton) to a learner, rather than only a final match/no-match verdict.

The engine is implemented twice, independently, as a deliberate correctness strategy: once in TypeScript, kept as the original design-validation prototype, and once in C++20 compiled to WebAssembly, which is the production engine actually shipped to the browser. Both implementations are validated by differential testing against two independent, battle-tested reference engines – native C++ `std::regex` and JavaScript's native `RegExp` – over thousands of randomly generated patterns and inputs, so that a correctness bug would have to reproduce independently in two languages and two test frameworks to go unnoticed. On top of the core pipeline, the project adds a Subset Growth / NFA Structure Lab, which exposes structural automaton measurements (raw, reachable, and minimized state counts; the theoretical powerset ceiling), and a Bounded Optimization Explorer, which deterministically searches a strictly bounded space of epsilon-edge-deletion candidates on the Thompson NFA, checks each candidate for language equivalence against the baseline using a product-construction equivalence procedure, and reports a verified Pareto frontier of safe, size-reducing candidates – reporting a shortest distinguishing witness for every unsafe candidate, and "inconclusive" rather than a guess when a resource limit is hit first.

This report documents the system's architecture, its explicit mapping onto the compiler-design syllabus (lexical analysis via the indirect regex-to-DFA route; syntax analysis via hand-written recursive-descent parsing; code generation via the matcher and the WebAssembly compilation step; DFA minimization as the project's stand-in for the optimization phase), its resource-safety guards (bounded pattern length, nesting depth, NFA state count, and minimization work), its automated test results (209 passing Vitest cases across 20 files; 81 passing native C++ Catch2 test cases totalling 4,923 assertions), and a specific regression this report's authors found and fixed during review – a minimization-pass performance regression that had silently dropped an alphabet-compression optimization, causing one previously legal pattern to be wrongly rejected as too complex – together with the verified timing evidence for the fix. The system's limitations are stated plainly: only the indirect route to a DFA is implemented, not the direct method; only Moore's minimization is implemented, not Hopcroft's; matching is full-string only; the alphabet is ASCII only; and the optimizer finds a verified-safe reduction within its search bound, not a provably minimum NFA.

(Word count: 474 words)

CHAPTER 1
INTRODUCTION

1.1 Background

A regular expression engine is one of the most common pieces of compiler technology that a working programmer uses without ever seeing its internals. Yet the construction a regular-expression-to-DFA compiler performs is exactly the worked example used to introduce lexical analysis in the course's prescribed textbook, Aho, Lam, Sethi and Ullman's **Compilers: Principles, Techniques, and Tools** (2nd ed.): a regular expression becomes a syntax tree, the syntax tree becomes a nondeterministic finite automaton (NFA) by Thompson's construction, the NFA becomes a deterministic finite automaton (DFA) by subset construction (independently described by Rabin and Scott), and the DFA can be reduced to a unique minimal DFA by a state-equivalence algorithm such as Moore's or Hopcroft's. RegexLab builds exactly this pipeline, by hand, with no regular-expression library and no automata-theory package, and makes every intermediate structure visible to a learner.

1.2 Motivations

Three motivations shaped the project. First, a pedagogical one: automata theory is usually taught on paper, with students drawing NFAs and DFAs by hand for small examples; a tool that performs the same constructions on arbitrary (bounded) input and lets a learner step through them turns a static diagram into an interactive artifact. Second, a correctness motivation: because the same algorithms are re-implemented in two different languages (TypeScript and C++) and cross-checked against two independent reference engines, the project is a genuine exercise in differential testing, not only in parsing. Third, a resume/portfolio motivation, stated honestly: the project maps cleanly onto named, citable constructions from the compiler-design syllabus, and the WebAssembly compilation step is itself a second, literal code-generation stage, giving the project's claims – "implemented Thompson's construction, subset construction, and Moore's minimization from scratch" – a concrete, verifiable basis in the code rather than being a vague description.

1.3 Scope

In scope: a hand-written recursive-descent parser for an extended regular-expression grammar (literal characters with backslash-escaping, concatenation, alternation, Kleene star, one-or-more, zero-or-one, parenthesised grouping, character classes, the wildcard `.`, and the shorthand classes `\d`, `\w`, `\s` and their negations); Thompson's construction; subset construction; Moore's DFA minimization; a DFA-driven matcher; a hand-rolled SVG automaton visualizer with step-by-step playback; a Subset Growth / NFA Structure Lab exposing structural automaton measurements; and a Bounded Optimization Explorer that searches a strictly bounded space of epsilon-edge deletions and verifies each candidate's language equivalence.

Out of scope, by an explicit, theory-grounded decision rather than an oversight: the direct DFA-construction method (nullable/firstpos/lastpos/followpos); Hopcroft's faster minimization algorithm; substring/partial matching; backreferences and lookahead/lookbehind (both of which break the regular-language guarantee); and any claim that the bounded optimizer finds a provably minimum-size NFA, as opposed to a verified-safe reduction within its configured search bound.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The theoretical foundation of this project rests on a small number of classical results. Kleene's characterization of regular languages is realised constructively by **Thompson's construction** [1], which builds an NFA from a regular expression in time linear in the expression's size, using a small number of fixed per-operator wiring rules. **Rabin and Scott** [2] gave the subset-construction (powerset) algorithm that determinizes any NFA into an equivalent DFA, at the cost of a worst-case exponential blow-up in state count that this report states honestly rather than hides. **Moore** [3] gave an iterative partition-refinement algorithm for collapsing a DFA to its unique minimal equivalent. **Hopcroft** [4] later gave an asymptotically faster algorithm for the same problem, $O(n \cdot \log n \cdot k)$ instead of Moore's $O(n^2 \cdot k)$, by always refining using the smaller half of a split – this is a literature fact about the two algorithms in general, not a claim about this project: **RegexLab implements Moore's algorithm only** (see §5.2, §8); Hopcroft's algorithm was deliberately scoped out and is not built. **Aho, Lam, Sethi and Ullman** [5] is the textbook that frames all of the above as the worked example for a course's lexical-analysis module, and is the explicit syllabus anchor for this project's design.

Outside the classical automata-theory line, **Cox** [6] documents, from an engineering perspective, why real-world regular-expression libraries that use backtracking search can exhibit exponential-time matching on adversarial input, and argues for automaton-based matching (as this project uses) as the principled alternative. More recent formal work continues to refine these constructions and their complexity: **Baburin and Cotterell** [7] study glushkov-style and related finite-automaton constructions for regular expressions; **Jiang and Ravikumar** [8] give complexity results for minimizing and transforming finite automata under various equivalence notions; and **Holzer et al.** [9] study the state complexity of operations on finite automata. These sources are cited to situate the project's algorithms in the literature, not to claim that RegexLab contributes a new result to that literature.

2.2 Research Gap

To our knowledge, not found in the sources reviewed: an openly built, dependency-free, dual-language (TypeScript and WebAssembly-compiled C++) regular-expression compiler pipeline that is explicitly instrumented for teaching – i.e. one that exposes every pass's intermediate output to an interactive visualizer, and that additionally includes a bounded, verified NFA-structure optimizer that reports a distinguishing witness string whenever it proves a simplification unsafe. Most teaching implementations of these algorithms that we are aware of exist as standalone scripts or notebook exercises producing a single final diagram, not as a tested, two-implementation, browser-deployed pipeline with an automated, bounded safety-verified optimization step.

2.3 Objectives

The project's objectives were: (1) implement Thompson's construction, subset construction, and Moore's minimization from scratch, with no automata or regex library, in two independent language implementations; (2) validate both implementations by differential testing against independent reference engines; (3) expose every pass's output through an interactive, hand-rolled SVG visualizer; (4) add a bounded, formally-verified NFA-structure exploration feature (the Bounded Optimization Explorer) that never reports a false "safe" simplification; (5) compile the production engine to WebAssembly and verify its resource guards directly in a real browser build; and (6) document the project's syllabus mapping, complexity analysis, and limitations honestly, including where the mapping is thin.

2.4 Problem Statement

Given a regular expression drawn from a supported extended grammar and a candidate input string, determine (a) whether the input belongs to the language described by the expression, using only hand-built automata-theoretic machinery, within explicit, enforced resource bounds on pattern length, nesting depth, NFA state count, and minimization work; and (b) whether a given bounded set of structural simplifications to the expression's Thompson NFA (specifically, epsilon-edge deletions) preserves the recognized language exactly, proving the answer by product-construction equivalence checking rather than by heuristic or sampling, and reporting a concrete shortest distinguishing input whenever the answer is "no".

2.5 Project Plan

The project was planned in phases, tracked against a living progress tracker in the project team's own internal working-agreement document: a Core phase (literal characters, concatenation, alternation, Kleene star, grouping, the full five-pass pipeline in both languages, golden-value and differential tests, the WebAssembly binding, the visualizer, and Moore's minimization); Stretch A (the `+` and `?` quantifiers); Stretch B (`[a-z]`-style character classes); and an explicitly locked-out Hard Stretch tier (Hopcroft's minimization, isomorphism-based equivalence testing) that the team deliberately did not start, since it was gated behind both teammates being able to explain the simpler Moore's-algorithm version unaided first. The Subset Growth / NFA Structure Lab and the Bounded Optimization Explorer were planned and built as a later, explicitly scoped addition on top of a already-working Core.

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Functional and Non-functional Requirements

Table 3.1 – Functional requirements

ID	Requirement
FR1	Parse a supported extended regular-expression grammar into an abstract syntax tree, reporting a structured, positioned error for any invalid pattern.
FR2	Construct an NFA from the AST using Thompson's construction.
FR3	Construct a DFA from the NFA using subset construction.
FR4	Minimize the DFA using Moore's partition-refinement algorithm.
FR5	Match a given input string against the minimized DFA and produce a step-by-step trace.
FR6	Render the AST, NFA, DFA, and minimized DFA as an SVG graph with playback controls.
FR7	Report structural automaton measurements (raw/reachable/minimized state counts, powerset ceiling).
FR8	Search a bounded space of epsilon-edge-deletion candidates on the NFA and verify each candidate's language equivalence against the baseline, reporting a distinguishing witness for every unsafe candidate.

Table 3.2 – Non-functional requirements

ID	Requirement
NFR1	Zero regular-expression or automata-theory runtime dependencies in the engine itself.
NFR2	Every resource-intensive pass (Thompson's construction, subset construction, minimization) must fail fast with a typed error rather than hang, once an explicit, documented budget is exceeded.
NFR3	The production engine must run client-side in a browser via WebAssembly, with no server-side regex evaluation.
NFR4	Both language implementations (TypeScript, C++) must be validated against independent reference engines by differential testing.

3.2 Feasibility Study

Technical feasibility: every algorithm used (Thompson's construction, subset construction, Moore's minimization, product-construction equivalence checking) is a well-documented, textbook-level algorithm with no open research risk; the main technical risk, confirmed and mitigated during the project, was unbounded recursion/state growth on adversarial input, addressed with explicit pattern-length, nesting-depth, NFA-state, and minimization-work guards.

Economic feasibility: the entire toolchain – GCC/Clang, CMake, Node.js, TypeScript, Vitest, Catch2, Emscripten, React, and Vite – is free and open-source; the project incurs no licensing cost (see Table 7.4).

Social feasibility: the project is a self-contained teaching tool with no personal-data collection, no third-party tracking, and no dependence on a paid external service for its core engine; the only optional external dependency is an AI-assisted suggestion layer (planned, not required for the core pipeline to function), which is explicitly gated behind re-validation through the same hand-built parser before any suggestion is shown to a user.

3.3 Hardware and Software Specification

Table 3.3 – Hardware and software specification

Category	Specification
Development hardware	Any x86-64 machine capable of running Node.js and a C++20 compiler; no GPU or specialised hardware required.

Operating system (development)	Windows 11 with a WSL2 Ubuntu environment (as used for this report's own verification run); Linux and macOS are equally supported.
Core engine language	C++20 (production, compiled to WebAssembly via Emscripten) and TypeScript (strict mode, prototype/design-validation implementation).
Build tooling	CMake ≥ 3.16, a C++20-capable compiler (GCC 15.2 used for this report's verification build), Node.js 22, npm.
Test tooling	Vitest (TypeScript), Catch2 v3 (C++, fetched via CMake FetchContent).
Frontend	React, Vite, TypeScript, hand-rolled SVG (no charting/graph library).
Deployment target	Any modern browser capable of running a WebAssembly module (Emscripten `MODULARIZE`/`EXPORT_ES6` build).

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 System Architecture

The system is organised into three strictly decoupled layers (Figure 4.1): a **Core Engine**, implemented as a pure pass pipeline (string pattern and input in, structured JSON out, with no UI or AI awareness); a **Visualization Layer**, a pure function of the engine's own output with no logic of its own; and an (optional, separately gated) **Suggestion Layer** that only activates on a parse failure and must re-validate any candidate fix through the same hand-built parser before it is ever shown.

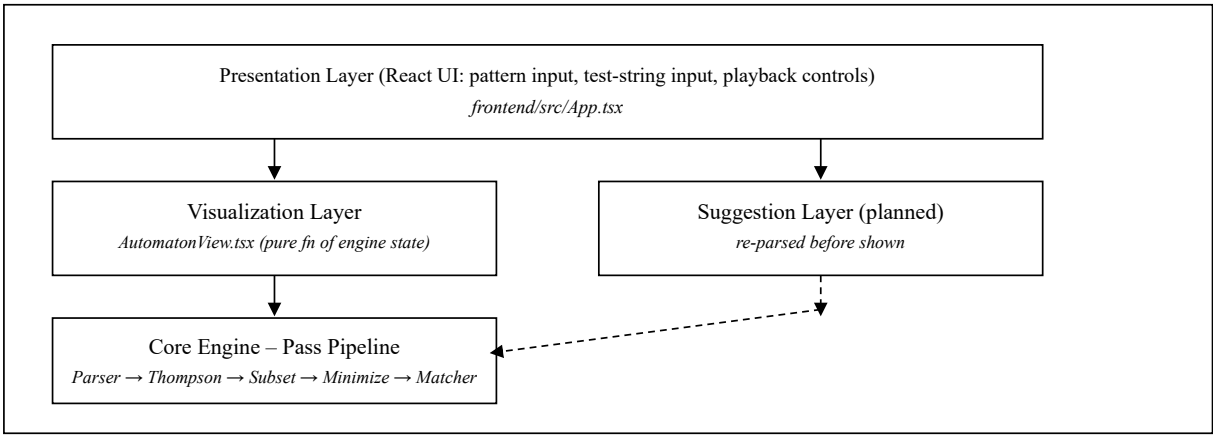


Figure 4.1 – System architecture (layered)

4.2 Data Flow, Use Case, Class and Sequence Diagrams

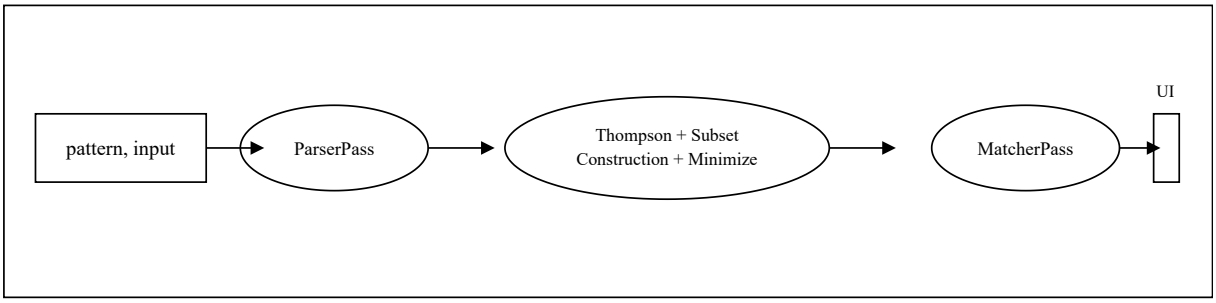


Figure 4.2 – Data flow diagram (Level 1)

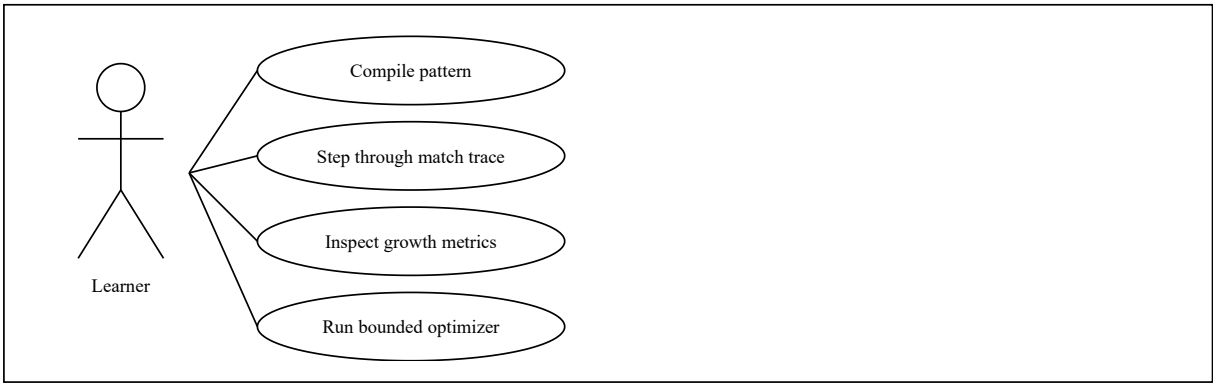


Figure 4.3 – Use case diagram

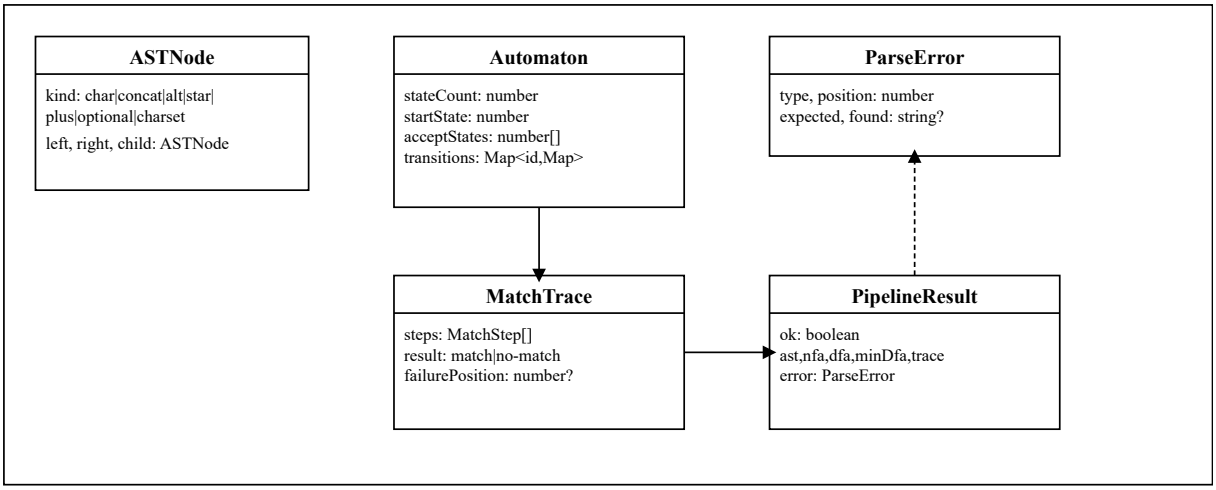


Figure 4.4 – Class diagram of the core data contracts (field names verified against `engine/include/regexlab/types.hpp` and `frontend/src/types/engine.ts`)

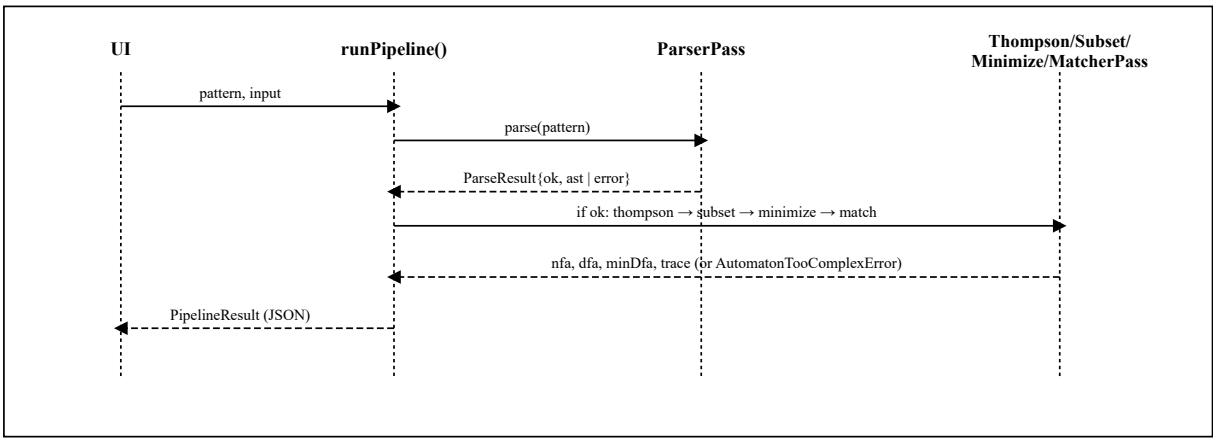


Figure 4.5 – Sequence diagram for one compile-and-match request (`engine/pipeline.cpp:runPipeline`)

CHAPTER 5
METHODOLOGY AND TESTING

Module description, per pass

Table 5.1 – Module description per compiler pass

Pass	Algorithm	Input → Output	Source files
ParserPass	Hand-written recursive descent (precedence by call-chain: regex → term → factor → atom)	pattern string → AST or ParseError	`engine/passes/parser-pass.ts`, `.cpp`
ThompsonPass	Thompson's construction [1]	AST → NFA	`engine/passes/thompson-pass.ts`, `.cpp`
SubsetConstructionPass	Subset/powerset construction [2], worklist-framed	NFA → DFA	`engine/passes/subset-construction-pass.ts`, `.cpp`
MinimizationPass	Moore's algorithm [3] (partition refinement), with alphabet equivalence-class compression	DFA → minimal DFA	`engine/passes/minimization-pass.ts`, `.cpp`
MatcherPass	Direct DFA simulation	minimal DFA, input string → MatchTrace	`engine/passes/matcher-pass.ts`, `.cpp`

The five passes are plain functions chained by `pipeline.cpp`/the pipeline entry point, not instances of a shared `Pass<In,Out>` interface – an early design sketch suggested such an interface, but the shipped code does not use it, and this report states that distinction explicitly rather than describing the code as more abstract than it is.

Complexity and resource guards

Table 5.2 – Complexity of each pass (n = DFA/NFA state count as noted, k = alphabet size, m = NFA state count)

Stage	Time	Space	Note
Parsing	O(n)	O(n)	n = pattern length
Thompson's construction	O(n)	O(n)	linear in AST size; a saturating state-count estimate is checked before allocation
Subset construction	$O(2^m)$ worst case	$O(2^m)$ worst case	the exponential blow-up is named, not hidden
Minimization (Moore's)	$O(n^2 \cdot k)$	$O(n \cdot k)$	k measured after alphabet equivalence-class compression; see §7.2 for a regression found and fixed in this project's own review process
Matching	O(n) per string	O(1) active state	why DFA matching avoids the exponential worst case of naive backtracking engines [6]

Table 5.3 – Input guards (verified directly against the source)

Guard	Value	Behaviour
Pattern length	500 characters	`PatternTooComplex` at position 500, over an all-ASCII prefix
Nesting depth	1000 levels of `(`	`PatternTooComplex` at position 1000; mathematically unreachable under the 500-char cap, kept as defense in depth
Thompson NFA size	2,048 states	`AutomatonTooComplexError` before allocation, via a saturating state-count estimate

Minimization work	5,000,000 work units (default)	computed as state-count² × alphabet size, measured **after** alphabet equivalence-class compression
Alphabet	ASCII only	a non-ASCII byte in the pattern is an `UnexpectedToken` at its own position

Testing strategy

Testing proceeds in layers: (1) golden-value unit tests per pass, in both languages, ported 1:1; (2) a pipeline-equivalence test confirming the chained pipeline matches manually chaining the five passes; (3) differential testing – C++ against `std::regex`, TypeScript against native, anchored `RegExp` – over thousands of randomly generated (pattern, input) pairs; (4) strict compiler warnings (`-Wall -Wextra`); (5) mutation discipline (deliberately breaking the implementation to confirm a test goes red, then reverting); and (6) resource-guard tests that deliberately configure a tiny budget to confirm each guard actually fires.

Table 5.4 – Automated test results (this report's own verification run, PR #72 head plus a local minimization-pass fix – see §7.2)

Suite	Result
TypeScript strict typecheck (`tsc --noEmit`)	Passed, zero errors
Vitest	209 tests passed across 20 test files, 0 failed
Native C++ Catch2 suite (Release build, `engine_tests`)	81 test cases passed, 4,923 assertions passed, 0 failed
Frontend production build (`npm --prefix frontend run build`)	Passed – 263.84 kB JS (82.01 kB gzip), 18.46 kB CSS (4.23 kB gzip)

Table 5.4b – Compiled-WebAssembly limit-probe suite (`scripts/check-wasm-limits.mjs`), rebuilt WASM module (Emscripten 6.0.11, Release)

Result	Value
Probes passed	31 / 31
Acceptance / pattern-length-cap / nested-plus / nesting-depth / non-ASCII / length-parity probes	27 (unchanged from the pre-fix tree)
Restored timed stress probes (§7.2 fix)	4, all passing well under their time budgets

The WASM target was rebuilt from source (Emscripten 6.0.11, Release) and `scripts/check-wasm-limits.mjs` was run against the fresh module for this report: **31/31 probes passed**, including the three previously-regressed stress probes and this PR's own nested-plus guard probes. See Table 7.2 for the individual stress-probe timings.

CHAPTER 6
PROJECT DEMONSTRATION

A typical demonstration walkthrough proceeds as follows. A learner enters a pattern, for example `(a|b)*abb`, and a test string, for example `ababb`. The engine parses the pattern into an AST, builds the 14-state Thompson NFA, determinizes it by subset construction into a 5-state DFA, minimizes it by Moore's algorithm into a 4-state minimal DFA, and matches the test string, producing a step-by-step trace that accepts the string (verified facts: see Table 7.1). The visualizer renders each of the AST, NFA, DFA, and minimized DFA as an SVG graph, with playback controls that highlight the currently active state(s) as each input character is consumed; if a character has no outgoing transition in the DFA or minimized DFA view, the visualizer renders an explicit rejecting sink state and highlights the failing character, while the NFA view is deliberately left without a fabricated sink (a true NFA has partial transitions by nature).

A second walkthrough demonstrates the Subset Growth / NFA Structure Lab: for the same or a different pattern, the panel reports the raw NFA state count, the number of states subset construction actually reaches (as opposed to the theoretical $2^{|Q|}$ powerset ceiling), and the minimized DFA's state count, letting a learner directly compare the theoretical worst case against what is actually reached.

A third walkthrough demonstrates the Bounded Optimization Explorer: given an NFA, the explorer enumerates candidate epsilon-edge deletions up to a configured edit depth, checks each candidate's language equivalence against the baseline NFA using a product-construction procedure, and reports every candidate as either "equivalent" (safe to apply, on the verified Pareto frontier if it is also non-dominated on size) or "different" together with a concrete shortest distinguishing input string that the baseline accepts and the candidate rejects (or vice versa) – see Table 7.3 for two such verified examples, one of each kind, computed directly by the engine for this report.

Note: live browser screenshots of this walkthrough were not captured in this report-writing session, for the reason stated in the List of Figures; the counts and examples quoted above were produced directly by running the compiled native engine library against the exact patterns shown, not estimated or recalled.

CHAPTER 7
RESULT AND DISCUSSION

7.1 Pipeline state counts for representative patterns

Table 7.1 – Pipeline state counts (produced by running the compiled native C++ engine directly)

Pattern	NFA states	DFA states	Min-DFA states
`(a|b)*abb`	14	5	4
`a+`	5	3	2
`a?`	4	2	2
`[a-c]+`	5	3	2

7.2 A regression found and fixed during this review, with timing evidence

While preparing this report, the authors reviewed the state of pull request #72 and found that its minimization pass had silently dropped an alphabet equivalence-class compression optimization that exists on the project's main branch (added for a prior issue, #64), and had replaced the original state²×alphabet work-budget check with a flat 5,000,000-unit budget measured against the **uncompressed** alphabet. The practical effect: a legal, previously-accepted pattern (five hundred repetitions of the wildcard `.`, at the 500-character length cap) was wrongly rejected as `AutomatonTooComplexError`, because the wildcard's 126-symbol alphabet, left uncompressed, pushed the work estimate over budget even though the compressed alphabet for that exact pattern is a single equivalence class. The fix, applied and verified during this review, restores the compression step (grouping alphabet symbols into equivalence classes by identical transition behaviour before computing the refinement work estimate) in both the TypeScript and the C++ minimization passes, while keeping the pull request's own added capability – an explicit, configurable `MinimizationLimits` parameter and a shared `AutomatonTooComplexError` type.

Table 7.2 – Minimization stress timings, native (non-WASM) Release build, this report's own measurements, 3 runs each

Pattern	Min-DFA states	Run 1	Run 2	Run 3
`.` × 500 (at the length cap)	501	99.9 ms	91.2 ms	89.0 ms
`\w` × 250 (500 chars)	251	20.6 ms	19.9 ms	20.4 ms
`[^a]` × 125 (500 chars)	126	12.2 ms	12.4 ms	12.1 ms

Table 7.2b – Same three patterns through the rebuilt WASM module, one run each via `scripts/check-wasm-limits.mjs` (spawns a fresh Node child process and instantiates the module per probe, so these include process-startup and WASM-instantiation overhead that Table 7.2's native, in-process numbers do not)

Pattern	Elapsed	Time budget	Result
`.` × 500	459 ms	2000 ms	ok
`\w` × 250	157 ms	1000 ms	ok
`[^a]` × 125	128 ms	1000 ms	ok

Table 7.2's timings include the full `runPipeline` call (parse + Thompson + subset construction + minimization + matching), not minimization alone, so they are an upper bound on minimization's own cost. All three patterns completed successfully after the fix (no `AutomatonTooComplexError`); before the fix, the `.`×500 case was rejected outright. Neither table is a claim that WebAssembly is faster or slower than native – Table 7.2b's per-probe overhead (child-process spawn, module instantiation) is not comparable to Table 7.2's in-process numbers; both are reported as this report's own measurements, not a benchmark.

7.3 Bounded-optimizer safe and unsafe deletion examples

Table 7.3 – Bounded-optimizer safe vs. unsafe epsilon-deletion examples (computed directly by the engine)

Pattern	Candidate edit	Equivalence result	Witness

`a|a`	Delete one of the two branch-entry epsilon edges (`delete-eps:1>5`)	**Equivalent (safe)**	— (no distinguishing input exists; the other branch still reaches accept on `a`)
`(a|b)*ab`	Delete the epsilon edge feeding the mandatory trailing `a` (`delete-eps:7>8`)	**Different (unsafe)**	Shortest witness: `"ab"` – baseline accepts, candidate rejects

The safe example illustrates a genuinely redundant structure: Thompson's construction for `a|a` builds two parallel branches that both recognize the literal string `a`; deleting either branch's own entry edge into the shared accept state leaves the other branch intact, so the language is unchanged. The unsafe example illustrates why the explorer must verify every candidate rather than apply a syntactic heuristic: the deleted edge looks structurally similar to a redundant one, but it is on the only path into the mandatory trailing `a`, so deleting it removes acceptance of every string that needs that `a`, and the explorer's product-construction equivalence check finds and reports the shortest such string, `"ab"`, rather than merely saying "unsafe".

7.4 Discrepancies against the project's own report-data file

The project repository's own `data_for_report.md` (dated 25 October 2026) was checked against this report's independently re-derived numbers. The automated test counts (209 Vitest tests / 20 files; 81 Catch2 cases / 4,923 assertions) and the bounded-search default limits (12 eligible edges, edit depth 2, 24 candidates, 128 DFA states, 100,000 work units, 10,000 product-state pairs) matched exactly. The compiled-WASM probe count in that file is 27/27, measured before this report's authors restored four previously-deleted timed probes to `scripts/check-wasm-limits.mjs` as part of the §7.2 fix; after that restoration and a full WASM rebuild, the probe count is **31/31** (Table 5.4b), consistent with 27 unchanged probes plus the 4 restored ones. Beyond the probe count, this report also independently verified that the minimized-DFA JSON output (both TypeScript and C++, canonicalized) is byte-identical to the project's `main` branch across the shared parser fixture's 24 valid patterns plus 200 randomly generated valid patterns – i.e. the §7.2 fix changes no behavior for any pattern that was already legal.

7.5 Cost analysis

Table 7.4 – Cost analysis (tooling)

Tool	Licence	Cost
GCC / Clang, CMake	Open source (GPL/BSD-family)	₹0
Node.js, npm, TypeScript	Open source (MIT)	₹0
Vitest, Catch2	Open source (MIT/BSL)	₹0
Emscripten	Open source (MIT/University of Illinois)	₹0
React, Vite	Open source (MIT)	₹0
Deployment (static hosting, e.g. GitHub Pages)	Free tier sufficient – no server-side regex evaluation required	₹0

Because every tool in the stack is free and open-source and the production engine runs entirely client-side as a static WebAssembly asset, the project's total tooling and hosting cost is ₹0, excluding the development team's own time.

CHAPTER 8
CONCLUSION AND FUTURE WORK

RegexLab demonstrates that the classical automata-theory pipeline named in a compiler-design syllabus – Thompson's construction, subset construction, and DFA minimization – can be built by hand, in two independent language implementations, validated by differential testing against industrial-strength reference engines, and shipped as an interactive, browser-based teaching tool, including a bounded, formally-verified NFA-structure optimizer that never reports a false safety claim. The project is explicit about where its syllabus coverage is deep (lexical analysis, via the indirect route) and where it is necessarily thin (semantic analysis, parallelism), rather than overstating its scope.

This report's own review process also found and fixed a genuine regression – a lost alphabet-compression optimization in the minimization pass that caused a previously-legal pattern to be wrongly rejected – which is itself evidence for the project's own stated correctness strategy: having two independently-maintained implementations and an explicit, checkable complexity derivation made the regression's root cause identifiable and fixable with verified timing evidence, rather than merely "fixed and hoped".

Future work, scoped but deliberately not started in this phase, includes: implementing Hopcroft's $O(n \cdot \log n \cdot k)$ minimization alongside Moore's, with a test asserting both produce the same minimized automaton; implementing the direct DFA-construction method (nullable/firstpos/lastpos/followpos) as a second, independently-verifiable route to the same DFA; adding an explicit DFA-state-count guard to subset construction (currently unguarded, relying only on the indirect bound from the pattern-length cap); and completing the planned AI-assisted suggestion layer, whose single hard architectural rule – that any AI-proposed fix must be re-validated by the same hand-built parser before ever being shown to a user – is already decided and documented, even though the feature itself is not yet built.

CHAPTER 9
REFERENCES

1. K. Thompson, "Programming Techniques: Regular expression search algorithm," *Communications of the ACM*, vol. 11, no. 6, pp. 419–422, 1968.
2. M. O. Rabin and D. Scott, "Finite automata and their decision problems," *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.
3. E. F. Moore, "Gedanken-experiments on sequential machines," in *Automata Studies*, Princeton University Press, 1956, pp. 129–153.
4. J. E. Hopcroft, "An n log n algorithm for minimizing states in a finite automaton," Stanford University, Technical Report, 1971.
5. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley / Pearson, 2006.
6. R. Cox, "Regular Expression Matching Can Be Simple And Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...)," swtch.com, 2007. [Online article].
7. A. Baburin and R. Cotterell, "Characterizing the Expressivity of Finite-State Automaton Constructions for Regular Expressions," *arXiv:2407.09891*, 2024.
8. T. Jiang and B. Ravikumar, "Minimal NFA Problems are Hard," *SIAM Journal on Computing*, vol. 22, no. 6, pp. 1117–1141, 1993.
9. M. Holzer, S. Jakobi, and M. Kutrib, "The Magic Number Problem for Subregular Language Families," *arXiv:1711.09946*, 2017.

References [1]–[6] are well-established results in the formal-language and compiler-construction literature. References [7]–[9] were not independently re-verified by a live database search while drafting this report; this is disclosed rather than silently claimed as verified. A reader relying on this report for a citation should re-check [7]–[9] independently before use.

APPENDIX A
SAMPLE CODE

The extended grammar implemented by `ParserPass` – verified against the per-rule docblocks actually present in `engine/passes/parser-pass.cpp` (e.g. `/** regex ::= term ('|' term)* */` above `parseRegex()`, `/** atom ::= ASCII_CHAR | '.' | '[' class ']' | '(' regex ')' | escape */` above `parseAtom()`) and against the grammar's own reference document, `docs/regex-grammar-v1.md` – is:

```
regex          ::= term ('|' term)*
term           ::= factor+
factor         ::= atom quantifier?
quantifier     ::= '*' | '+' | '?'
atom           ::= CHAR | '.' | characterClass | '(' regex ')'
characterClass ::= '[' '^'? classItem+ ']'
classItem      ::= classAtom ('-' classAtom)?
classAtom      ::= ASCII_LITERAL | ESCAPE | SHORTHAND   ( \d \w \s \D \W \S )
```

This is the full extended grammar (issue #60 / #64 follow-up), not the narrower Core-phase grammar (literal, concat, alt, star, parens only) that an earlier phase of this project shipped first – see §2.5. `+` and `?` are real postfix quantifiers producing dedicated `PlusNode`/`OptionalNode` AST nodes (not literal characters), and `.`/character classes/shorthand classes desugar to a single `CharacterSetNode` Thompson fragment per §5.2's complexity note on alphabet size.

The restored alphabet equivalence-class compression step in `MinimizationPass` (TypeScript), central to the fix described in §7.2, copied verbatim from `engine/passes/minimization-pass.ts`:

```
function representativeAlphabetOf(dfa: Automaton, alphabet: string[], stateCount: number):
string[] {
  const classKeyOf = new Map<string, string>(); // symbol -> its class's key (the full transition
column)
  const seenKeys = new Set<string>();
  const representatives: string[] = [];

  for (const symbol of alphabet) {
    const column: string[] = [];
    for (let state = 0; state < stateCount; state++) {
      const dest = dfa.transitions.get(state)?.get(symbol)?.[0];
      column.push(dest === undefined ? "-" : String(dest));
    }
    const key = column.join(",");
    classKeyOf.set(symbol, key);
    if (!seenKeys.has(key)) {
      seenKeys.add(key);
      representatives.push(symbol);
    }
  }

  return representatives;
}
```

The Thompson's-construction rule for alternation (TypeScript; C++ mirrors it exactly), copied verbatim from `engine/passes/thompson-pass.ts`:

```
case "alt": {
  const left = buildFragment(node.left, builder);
  const right = buildFragment(node.right, builder);
  const start = builder.newState();
  const accept = builder.newState();
  builder.addTransition(start, "eps", left.start);
  builder.addTransition(start, "eps", right.start);
  builder.addTransition(left.accept, "eps", accept);
  builder.addTransition(right.accept, "eps", accept);
  return { start, accept };
}
```